

SELENIUM WEBDRIVER

Ejemplo Practico con Selenium

Elaborado por:

David Quesada Calderón.

Fecha:

03 de septiembre de 2026

Herramienta:

Selenium 4.50.0

Ejemplo práctico de una prueba automatizada con Selenium

1. Objetivo

El objetivo de este ejercicio es crear y ejecutar pruebas automatizadas utilizando Selenium WebDriver, Python y Pytest.

Para el ejercicio se automatizarán diferentes acciones realizadas por un usuario.

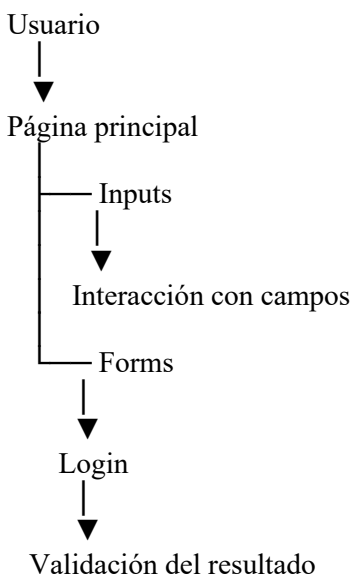
Durante el ejercicio se aprenderá a:

- Configurar un proyecto de Selenium con Python.
- Configurar Selenium WebDriver.
- Utilizar Pytest para ejecutar pruebas.
- Identificar elementos mediante diferentes tipos de locators.
- Interactuar con elementos de una página web.
- Utilizar el patrón Page Object Model (POM).
- Utilizar fixtures de Pytest.
- Realizar validaciones mediante assert.
- Automatizar formularios.
- Validar escenarios positivos y negativos.

2. Escenario de prueba

Para este ejercicio se utilizará una página web de práctica con diferentes secciones.

El flujo general será:



3. Estructura del proyecto

El proyecto utiliza el patrón **Page Object Model (POM)**.

selenium-qa-practice/

```
|
|
|— pages/
|   |— home_page.py
|   |— inputs_page.py
|   |— forms_page.py
|
|— tests/
|   |— test_home_page.py
|   |— test_inputs.py
|   |— test_forms.py
|
|— conftest.py
|— requirements.txt
```

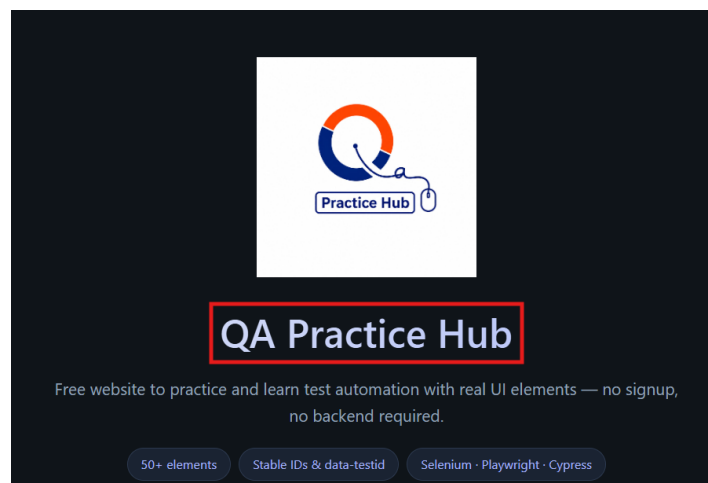
4. Casos de prueba

Prueba #1: Validar la página principal

Validar que la página principal se cargue correctamente.

Pasos:

1. Abrir el sitio.
2. Validar que el título principal sea: **QA Practice Hub**



Resultado esperado

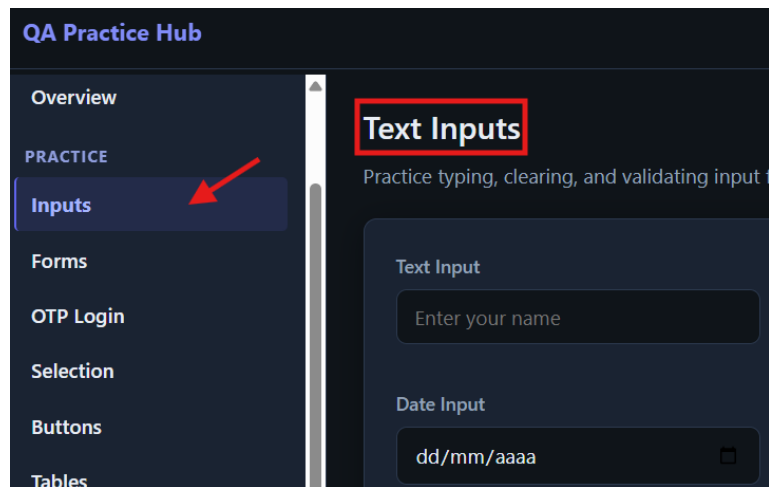
La prueba debe finalizar correctamente cuando el título coincida con el valor esperado.

Prueba #2: Validar navegación hacia Inputs

Validar que el usuario pueda acceder a la sección Inputs.

Pasos:

1. Click en Inputs.
2. Validar que aparezca el título: Text Inputs



Resultado esperado

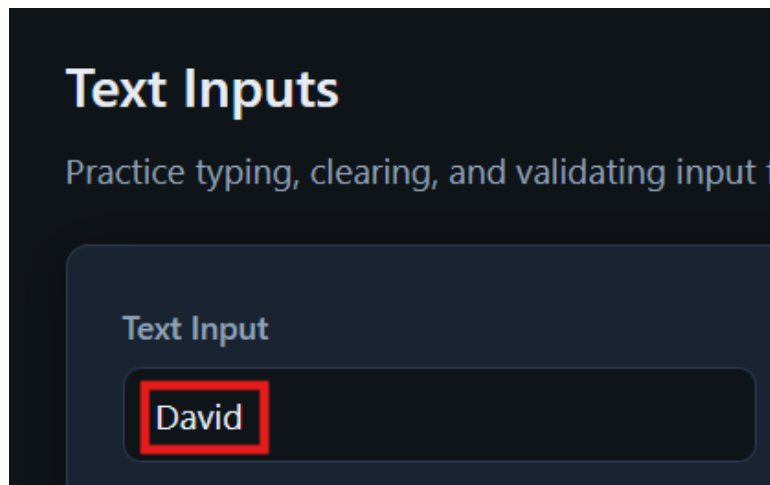
La navegación debe realizarse correctamente y debe mostrarse la sección correspondiente.

Prueba #3: Interacción con un campo de texto

Validar las principales operaciones sobre un campo de texto.

Pasos:

1. Ingresar texto.
2. Obtener valor.
3. Limpiar campo.



Resultado esperado

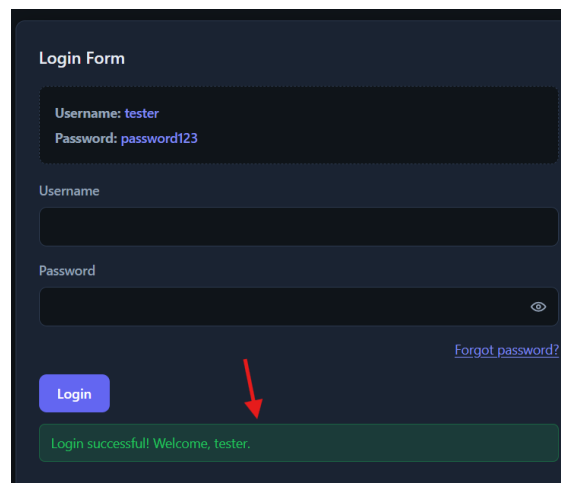
El texto ingresado debe almacenarse correctamente en el campo y, posteriormente, el campo debe poder limpiarse correctamente.

Prueba #4: Login exitoso

Validar el funcionamiento del formulario de Login utilizando credenciales válidas.

Pasos:

1. Ingresar el Username
2. Ingresar el Password
3. Click en Login



Resultado esperado

Debe mostrarse: **Login successful! Welcome, tester.**

5. Resultado de las pruebas

Al ejecutar Pytest se obtiene un resumen de las pruebas ejecutadas.

```

(venv) PS C:\Users\david\OneDrive\Imágenes\Proyecto Selenium\selenium-qa-practice> pytest
===== test session starts =====
platform win32 -- Python 3.13.6, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\david\OneDrive\Imágenes\Proyecto Selenium\selenium-qa-practice
collected 5 items

tests\test_forms.py . [ 20%]
tests\test_home_page.py .. [ 60%]
tests\test_inputs.py .. [100%]

===== 5 passed in 18.43s =====

```

6. Conclusión

En este ejercicio se creó una base para realizar pruebas automatizadas utilizando Selenium, Python y Pytest.

Se configuró un proyecto utilizando Page Object Model y se automatizaron diferentes escenarios sobre una página web de práctica.

Durante el ejercicio se trabajó con:

- Selenium WebDriver.
- Pytest.
- Locators.
- Click.
- Send Keys.
- Clear.
- Obtención de texto.
- Obtención de atributos.
- Page Object Model.
- Fixtures.
- Assertions.
- Casos positivos y negativos.

Esta estructura constituye una base para continuar desarrollando pruebas automatizadas más completas, incorporando posteriormente esperas, elementos dinámicos, manejo de ventanas, alertas, iframes y otros componentes habituales en la automatización web.